



Lab 10 — Red-Team an AI System Before Deployment

Topic 03: Governance Controls Across the AI Lifecycle

Objective: Develop pre-deployment assurance by red-teaming an AI application for injection, access-control and data-leakage weaknesses, and record the evidence.

Goal: Testing that a model is accurate is not assurance. You will run an adversarial pre-deployment test against the FauxBank training sandbox, find the vulnerability classes that matter for an AI-enabled banking application, and produce the test evidence a deployment gate requires.

What you'll build: A pre-deployment red-team report: findings with severity, evidence and OWASP mapping, extended with the AI-specific test cases and a go/no-go recommendation.

Tools and data: FauxBank pentest sandbox (<https://pentest-fauxbank.vercel.app/>), Hacklab (<https://alfredang.github.io/ethnicalhacking/>), red-team report template

Data files in this lab

- `data/red-team-report-template.md`

Step-by-step

1. Open <https://pentest-fauxbank.vercel.app/> and read the disclaimer: this is a training sandbox with fictional data. Never run these techniques against a real system without written authorisation.
2. Work through the Guided Pentest scenarios. Record each finding as you go — what you did, what happened, and what it proves about the control that failed.
3. Test for IDOR (Insecure Direct Object Reference). Change an identifier in a request and observe whether you reach another user's record. Note why this matters doubly for an AI agent, which iterates far faster than a human.
4. Test for broken access control and injection using the guided scenarios, and record the evidence for each: the input, the response, and the impact.
5. Run the Simulated Scanner and compare its findings to yours. Note which issues you found that the scanner missed and which the scanner found that you missed — this is the argument for combining automated and manual assurance.
6. Open <https://alfredang.github.io/ethnicalhacking/> and run the reconnaissance and enumeration labs. Type help to see the available commands and objectives to see the checklist for each lab.

[help](#) · [objectives](#)

7. Complete the scanning and enumeration labs in Hacklab, and record how much an attacker learns before touching the application. Map this to what an AI agent's own tool calls could reveal if logged insecurely.
8. Now extend the test plan with the AI-specific cases the tools do not cover: direct prompt injection, indirect injection through retrieved content, system-prompt extraction, unsafe tool invocation, and PII leakage in outputs.
9. For each AI-specific test case, write the test input, the expected safe behaviour and the observed behaviour, using the NovaAssist brief from Lab 3 as the system under test.
10. Generate the report using FauxBank's Report Generator, then extend it with your AI test cases, severity ratings and a clear go/no-go recommendation with the conditions attached to a go.

Test it

Your report contains findings from the sandbox with evidence and OWASP mapping, plus at least five AI-specific test cases the automated scanner could not produce. Your recommendation states conditions, not just a verdict.

Training use only. The organisation, data and individuals in this lab are fictional. Security techniques taught here must only be applied to systems you own or have written authorisation to test.